Guía de Actividades **PrácticoExperimentales** Nro. 001

1. Datos Generales

Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	Uno
Resultado de aprendizaje de la unidad	R1. Disena arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	1
Título de la Práctica	Uso de Wokwi y Aruino IDE en Microcontroladores AVR
Nombre del Estudiante	Viviana Elizabeth Córdova Celi
Fecha	09 de abril de 2026
Horario	11h30 a 13h30
Lugar	Aula
Tiempo planificado en el Sílabo	2

2. Objetivo(s) de la Práctica:

- Comprender y aplicar el entorno de desarrollo Wokwi para simulación de circuitos con microcontroladores AVR.
- Familiarizarse con la estructura básica de un programa en Arduino IDE, identificando las funciones setup() y loop(), declaración de variables, tipos de datos, constantes y operaciones bitwise.

3. Materiales y reactivos:

- 1 Protoboard
- 1 Arduino UNO
- 1 LED Rojo 5mm
- 1 Resistencia 220 ohmios
- 1 Cable USB
- Cables puente

4. Equipos y herramientas

- Computadora
- Arduino IDE
- Wokwi (wokwi.com)

5. Resultados Esperados

Paso 1: Ingresar a Wokwi y crear un proyecto Arduino UNO. Explorar la interfaz del simulador.

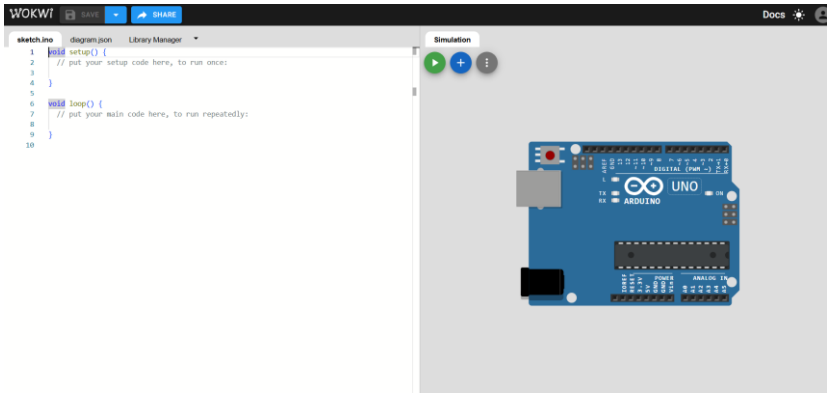


Imagen 1: explorar la interfaz de simulación en Wokwi

Paso 2: Montar circuito básico en protoboard virtual: LED en pin D13 con resistencia 220 ohmios a GND.

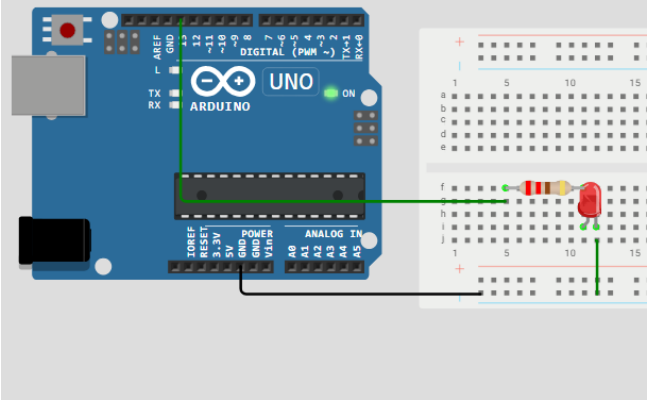


Imagen 2: Diseño de circuito en Wokwi

Paso 3: Escribir programa que declare variables de diferentes tipos (int, byte, long, float, bool), realice operaciones bitwise (AND, OR, XOR, NOT, shift) y las aplique para controlar el LED.

sketch.ino ● diagram.json Library Manager

```
const int LED_PIN = 13;
byte estadoLed = 0;
int contador = 0;
long enteroLargo = 1000;
float decimal = 3.14;
bool estado = true;

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);

  Serial.println("Operaciones Bitwise:");

  byte a = 0b11001100;
  byte b = 0b10101010;

  Serial.print("AND: ");
  Serial.println(a & b, BIN);

  Serial.print("OR: ");
  Serial.println(a | b, BIN);

  Serial.print("XOR: ");
  Serial.println(a ^ b, BIN);

  Serial.print("NOT a: ");
  Serial.println(~a, BIN);

  Serial.print("Shift izquierda: ");
  Serial.println(a << 1, BIN);

  Serial.print("Shift derecha: ");
  Serial.println(a >> 1, BIN);

  estadoLed = estadoLed | (1 << 0);
}

void loop() {
  estadoLed = estadoLed ^ 0b00000001;

  if ((estadoLed & 1) == 1) {
    digitalWrite(LED_PIN, HIGH);
  } else {
    digitalWrite(LED_PIN, LOW);
  }
  delay(500);

  contador = (contador + 1) % 8;

  Serial.print("Shift (1 << contador): ");
  Serial.println(1 << contador, BIN);

  delay(500);
}
```

Imagen 3: generación de código para operaciones bitwise

Paso 4: Verificar funcionamiento en simulador Wokwi observando el LED.

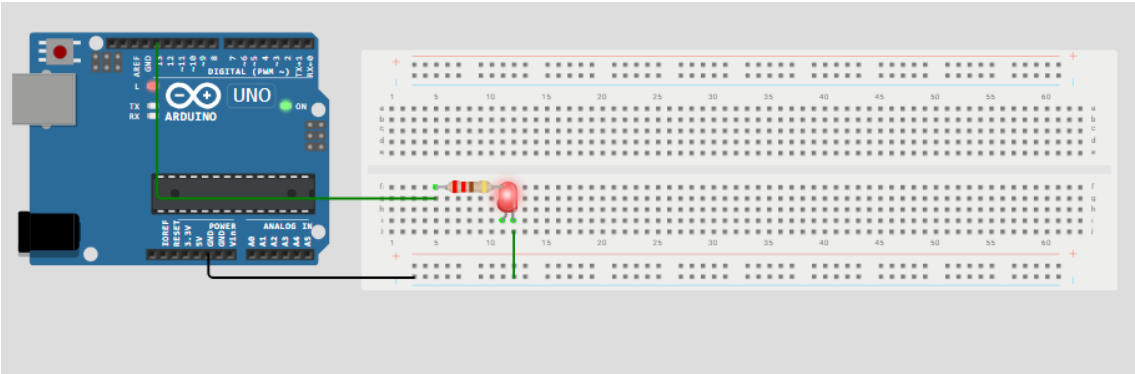


Imagen 4: Simulación en Wokwi

Paso 5: Replicar en Arduino IDE y comparar diferencias en estructura y compilación.

Arduino UNO

sketch_apr9a.ino

```
const int LED_PIN = 13;
byte estadoLed = 0;
int contador = 0;
long enteroLargo = 1000;
float decimal = 3.14;
bool estado = true;

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);

  Serial.println("Operaciones Bitwise:");

  byte a = 0b11001100;
  byte b = 0b10101010;

  Serial.print("AND: ");
  Serial.println(a & b, BIN);

  Serial.print("OR: ");
  Serial.println(a | b, BIN);

  Serial.print("XOR: ");
  Serial.println(a ^ b, BIN);

  Serial.print("NOT a: ");
  Serial.println(~a, BIN);

  Serial.print("Shift izquierda: ");
  Serial.println(a << 1, BIN);

  Serial.print("Shift derecha: ");
  Serial.println(a >> 1, BIN);

  estadoLed = estadoLed | (1 << 0);
}

void loop() {
  estadoLed = estadoLed ^ 0b00000001;

  if ((estadoLed & 1) == 1) {
    digitalWrite(LED_PIN, HIGH);
  } else {
    digitalWrite(LED_PIN, LOW);
  }
  delay(500);

  contador = (contador + 1) % 8;

  Serial.print("Shift (1 << contador): ");
  Serial.println(1 << contador, BIN);

  delay(500);
}
```

Imagen 5: Código copilado en Arduino IDE

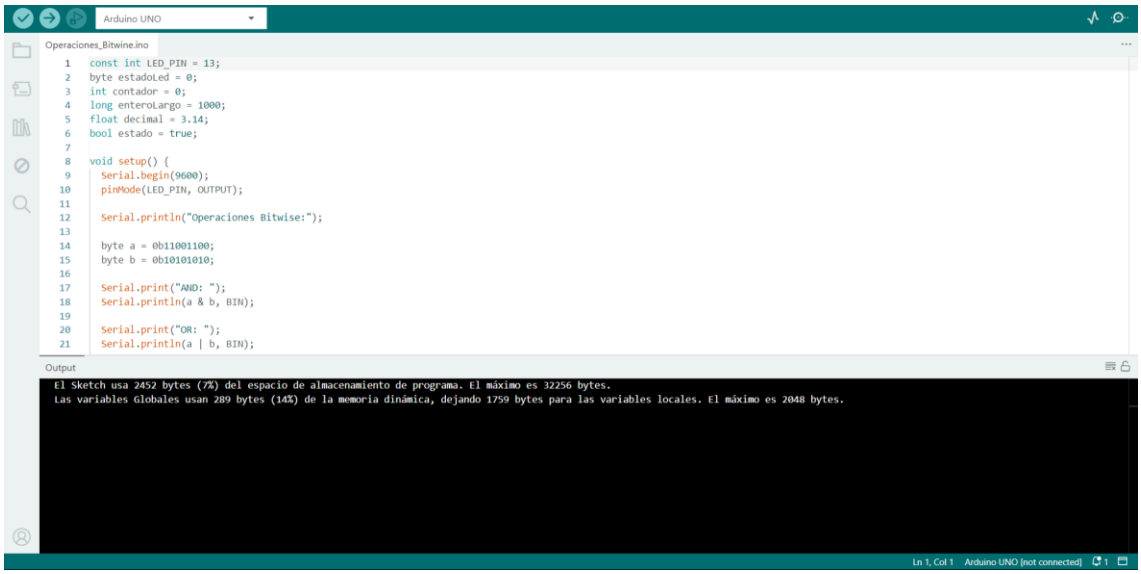


Imagen 6: Copilar código en Arduino IDE

Paso 6: Documentar resultados y análisis comparativo entre entornos

Wokwi: es una plataforma que contiene una amplia gama de componentes de hardware lo que permite simular diferentes circuitos, lo que permite ver la funcionalidad de los componentes antes de aplicarlos en componentes físicos.

Por otro lado, en entorno de Arduino IDE, es una plataforma que permite ya manipular los componentes físicos de hardware, es decir ya tratar con las placas de Arduino.

Link de Wokwi: https://wokwi.com/projects/460834319281314817

Algoritmo

```
ALGORITMO APE 01 - Operaciones Bitwise
INICIO
  LED_PIN <-- 13
estadoLed <-- 0   contador
<-- 0
  CONFIGURAR LED_PIN como SALIDA
  INICIAR serial 9600
  IMPRIMIR resultados de AND, OR, XOR, NOT, Shift
estadoLed <-- estadoLed OR (1 << 0)   REPETIR
    estadoLed <-- estadoLed XOR 0b00000001     SI
(estadoLed AND 1) = 1 ENTONCES ENCENDER LED
    SINO APAGAR LED
ESPERAR 500ms
    contador <-- (contador + 1) MOD 8
    IMPRIMIR (1 << contador)
    ESPERAR 500ms
  FIN REPETIR
FIN
```

6. Preguntas de Control:

1. Explique que es una operación bitwise y su importancia en microcontroladores.

   Una operación bitwise, realizan una seria de operaciones de un solo operan un bit en paralelo (0 y 1). La importancia de estos operadores dentro de los microcontroladores, reside en la manipulación de registros y variables binarias del microcontrolador, lo que permite realizar operaciones lógicas cobre cada bit.

2. Diferencias entre tipos byte, int, long, float en Arduino con ejemplos.

   Los tipos de datos en Arduino se diferencias por su tamaño en memoria.

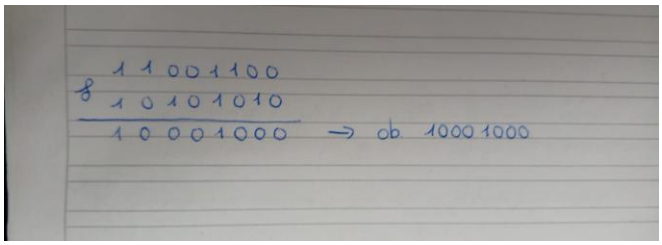
Tipo de dato	Diferencias	Ejemplos
byte	Almacena un valor numérico de 8 bits, teniendo un rango de 0-255, lo que indica que solo se puede almacenar enteros positivos.	Byte x = 200;

int	Almacena un valor entero de 16 bits, con un rango de 3.4028235E+38 a -3.4028235E+38, puede almacenar tanto números positivos como negativos.	int countUp = 0
		void setup() {
		Serial.begin(9600);
		}
		void loop() {
		countUp++
		Serial.println(countUp
		delay(1000);
		}

long	Valor entero almacenado en 32 bits con un rango de 2,147,483,647 a -2,147,483,648, lo que indica que puede almacenar mayor cantidad que los enteros.	long x = 1000000;

float	Tipo como flotante almacenado en 32 bits con un rango de 3.4028235E+38 a -3.4028235E+38. Puede almacenar numero con decimales, muy grandes o muy pequeños	float myfloat;
		float sensorCalbrate = 1.117;
		
		int x;
		int y;
		float z;
		
		x = 1;
		y = x / 2;
		z = (float)x / 2.0;

3. Resultado de 0b11001100 & 0b10101010 mostrando procedimiento.



4. Uso del desplazamiento de bits en control de periféricos.

   el desplazamiento de bits (<<, >>) es fundamental en el control de periféricos para la manipulación de registros de hardware, permitiendo la activación y desactivación bits.

5. Ventajas y desventajas de Wokwi vs hardware físico.

   Una de las mayores ventajas de Wokwi es que permite simular circuitos en tiempo real, permite experimentar las diferentes configuraciones y conceptos electrónicos sin el riesgo de dañar dispositivos físicos.

   Por otro lado, una de las desventajas es que en algunos casos las simulaciones no replican con fidelidad el comportamiento de hardware físico, lo que pude provocar fallos cuando ya se trabaja con los componentes físicos

7. Evaluación

8. Bibliografía

1. Arduino Reference. Bitwise Operators. arduino.cc/reference
2. Mazidi, M. A. (2020). The 8051 Microcontroller. Pearson.
3. Barr, M. (2020). Programming Embedded Systems. O'Reilly.

9. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	